

Full Stack Engineer Assignment

Interactive US Retail Locations Map

Time Limit: 2 hours

1. Overview

You will build a performant, interactive map of the United States that visualizes retail store locations from a dataset we provide. The map must gracefully handle a dataset of tens of thousands of points by adapting what it shows based on the zoom level and the current map viewport.

This task evaluates your ability to: design a clean backend API, integrate with a third-party mapping service (Google Maps), implement efficient data fetching for large geospatial datasets, and handle clustering / level-of-detail rendering on the client.

2. What We Provide

- **[CSV dataset](#)** — A sanitized export containing store locations (brand initial, latitude, longitude, state, city, zipcode, status, type, channel). Approximately 150k+ rows covering all US states. [Download the CSV dataset here](#)
- **Google Maps API key** — A fully working key scoped to Maps JavaScript API, Places, and Geocoding. Do NOT commit it; read it from an environment variable. You can use the key `AlzaSyDAnh7rtiiJWBSiT6f6eZit0qce9GNP0bc` for this task.
- **Brand logo assets** — A small set of placeholder brand logos (PNG) for use at the deepest zoom level. Map brand initial → logo by simple lookup.

3. Functional Requirements

3.1 Three Zoom Tiers

The map must switch between three distinct visualization modes based on the current zoom level:

- **Tier 1 — Country view (zoomed all the way out):** Show one circular marker per US state, positioned at the state's centroid, displaying the total count of stores in that state (e.g., "CA 12k", "TX 11k", "FL 9.5k"). No individual store points should be fetched at this level.
- **Tier 2 — Regional / mid-zoom:** Show clustered markers. Each cluster displays the count of stores within its radius. Clicking or zooming further should expand clusters.
- **Tier 3 — Street / deep zoom:** Show individual store markers rendered with their brand logo. Each marker should be clickable and reveal basic store info (brand, address, status, type) in a popup/info window.

3.2 Viewport-Based Fetching

This is the most important performance requirement. The backend must only return data for the currently visible map viewport — never the entire dataset.

- **On every significant map move/zoom**, the frontend sends the current viewport bounds (NE + SW lat/long, plus zoom level) to the backend.
- **The backend responds with** only the points (or aggregated clusters/state counts) that fall inside those bounds, appropriate for the current zoom tier.
- **Debounce map events** (e.g., 200–300 ms) so you are not firing a request on every pixel pan.
- **Avoid refetching** when the user pans within already-loaded bounds. A simple client-side cache keyed by rounded bounds is sufficient.

3.3 Filters (Optional but Recommended)

If time allows, implement a filter sidebar supporting at least: state, brand (by initial), and status. Filters must be applied server-side and respect the current viewport.

4. Technical Requirements

4.1 Stack

- **Backend:** Node.js (Express / Fastify / NestJS) OR Python (FastAPI / Flask). Your choice.
- **Frontend:** React (preferred) with Google Maps JavaScript API. Use `@react-google-maps/api` or `@vis.gl/react-google-maps`.
- **Data store:** PostgreSQL with PostGIS is ideal, but SQLite + a spatial index, or even an in-memory R-tree / quadtree, is acceptable given the time limit.
- **Clustering:** Supercluster on the server OR MarkerClusterer on the client — either is acceptable. Document your choice.

4.3 Non-Functional

- API key must be read from `.env`. Do not hardcode or commit it.
- Viewport queries should return in under 500ms for any reasonable viewport.
- Code should be organized, typed where possible (TypeScript preferred), and reasonably commented.
- Handle empty states and loading indicators gracefully.

5. Deliverables

1. A public GitHub repository (or zipped archive) containing backend + frontend code.
2. A README.md with: setup steps, how to run, environment variables, tech choices, and trade-offs you made given the time limit.
3. A 2–4 minute screen recording (Loom or similar) walking through your solution: the three zoom tiers in action, viewport fetching visible in network tab, and one thing you would improve with more time.

7. Evaluation Criteria

- **Correctness (40%)** — Do the three zoom tiers work? Does viewport fetching actually restrict what is returned?
- **Performance & Architecture (30%)** — Sensible API design, debouncing, caching, spatial indexing choice.
- **Code Quality (20%)** — Readability, structure, typing, error handling.
- **Communication (10%)** — Clear README, honest trade-off documentation, good walkthrough video.

8. Ground Rules & Notes

- You may use any open-source library, AI coding assistant, or Stack Overflow. Document significant AI usage in your README.
- Partial completion with clean code beats a buggy full implementation. If you run out of time, ship what works and document the rest.
- If any requirement is ambiguous, make a reasonable assumption and note it in the README.
- Do not redistribute or repost the provided dataset or API key.

9. Dataset Schema (CSV Columns)

The shared CSV contains the following columns:

- **id** — unique identifier for the store
- **brand_initial** — single-letter abbreviation of the brand name (use this to map to logo)
- **latitude, longitude** — store coordinates
- **state** — State name
- **city** — city name
- **status** — store status (e.g., Active, Closed, Planned)

Good luck — we look forward to seeing your approach!